

Nama : Cecillia Yowien Arthamevia
NIM : 224308031
Kelas : TKA-6B
Mata Kuliah Praktikum Kontrol Cerdas

Tugas Kontrol PID

Dalam proses pengerjaan tugas kontrol PID pada mata kuliah Praktikum Kontrol Cerdas menggunakan *software* Matlab dan Arduino IDE, adapun beberapa tahapan pengerjaannya diantaranya adalah sebagai berikut :

1. Kode Program *Transfer Fuction* :

```
clear
clc
% Parameter Motor
R = 0.4;
L = 2.7;
J = 0.0004;
B = 0.0022;
Ke = 0.015;
Kt = 0.05;

% Numerator dan Denominator
num = Ke;
den = [L*J (L*B + R*J) (R*B + Ke*Kt)];

% Transfer Function (Open-loop)
Gs = tf(num, den);

% PID parameters
kp = 53.7208;
Ti = 0.73245;
Td = 0.17711;
ki = kp / Ti;
kd = kp * Td;

% Buat controller PID
PID = pid(kp, ki, kd);

% Closed-loop transfer function (Negative feedback, unity)
T = feedback(PID * Gs, 1);

% Step response
step(T)
title('Step Response with PID Controller')
grid on
```

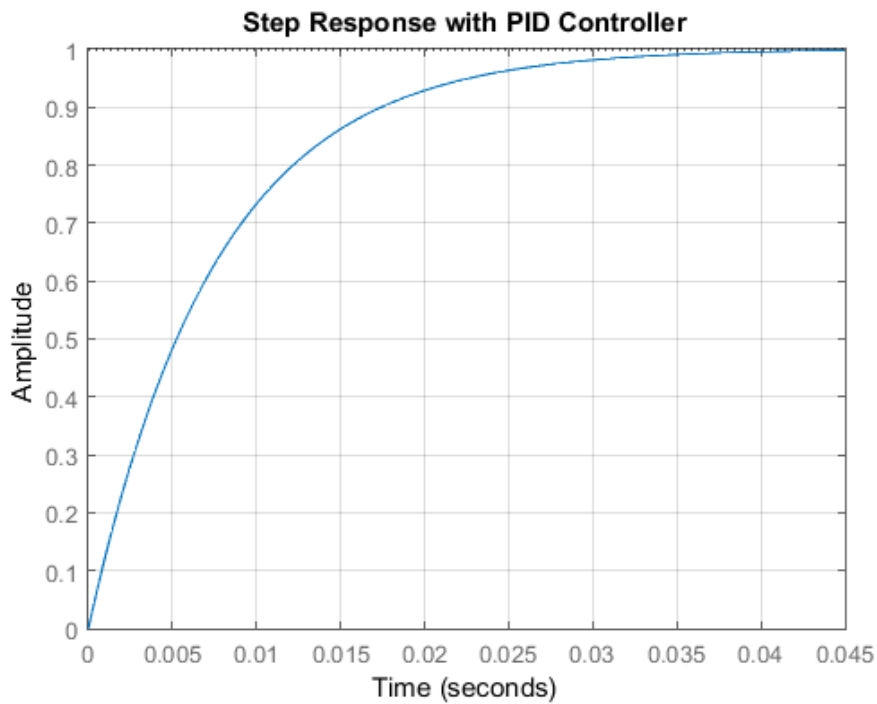
```

% Hitung karakteristik step response
info = stepinfo(T)

% Tampilkan hasil karakteristik
fprintf('\nKarakteristik Step Response:\n');
fprintf('Rise Time    : %.4f s\n', info.RiseTime);
fprintf('Peak Time     : %.4f s\n', info.PeakTime);
fprintf('Overshoot     : %.2f %%\n', info.Overshoot);
fprintf('Settling Time : %.4f s\n', info.SettlingTime);

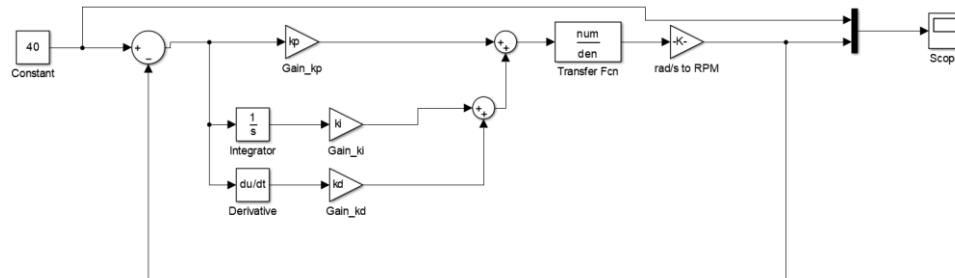
```

2. *Output Hasil Running Program :*



Info =	Karakteristik <i>Step Response</i> :
<i>Rise Time</i> : 0.0166	<i>Rise Time</i> : 0.0166 s
<i>Settling Time</i> : 0.0294	<i>Peak Time</i> : 0.0554 s
<i>Settling Min</i> : 0.9002	<i>Overshoot</i> : 0.10 %
<i>Settling Max</i> : 1.0010	<i>Settling Time</i> : 0.0294 s
<i>Overshoot</i> : 0.1002	

3. Model Simulink :



Hasil Simulink :



4. Kode Program Arduino IDE :

```
#include <WiFi.h>
#include <PubSubClient.h>
// =====
// KONFIGURASI WIFI & MQTT
// =====
const char* ssid = "TKA 6Barokah";
const char* password = "65432100";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
WiFiClient espClient;
PubSubClient client(espClient);
// =====
// PIN KONFIGURASI
// =====
const int pinMotorPWM = 5;
const int pinREn = 21;
```

```

const int pinLEn = 22;
const int encoderPinA = 34;
const int encoderPinB = 35;
// =====
// VARIABEL PID & MOTOR
// =====
kp = 53.7208;
Ti = 0.73245;
Td = 0.17711;
ki = kp / Ti;
kd = kp * Td;
float error = 0, lastError = 0, integral = 0;
float targetSpeedKmh = 0;
float actualSpeedKmh = 0;
int pidPWM = 0;
int throttleLevel = 0;
String motorDirection = "forward"; // Default arah
// =====
// ENCODER
// =====
volatile long encoderCount = 0;
unsigned long lastSpeedCheck = 0;
void IRAM_ATTR encoderISR() {
    encoderCount++;
}
// =====
// WIFI DAN MQTT SETUP
// =====
void setup_wifi() {
    Serial.print("Connecting to WiFi ");
    Serial.print(ssid);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.println("WiFi connected, IP: " + WiFi.localIP().toString());
}

void reconnect() {
    while (!client.connected()) {
        Serial.print("Connecting to MQTT...");
        String clientId = "ESP32Client-";
        clientId += String(random(0xffff), HEX);
        if (client.connect(clientId.c_str())) { // Connect tanpa user/password
            Serial.println("connected!");
        }
    }
}

```

```

        client.subscribe("motor/throttle");
        client.subscribe("motor/direction");
    } else {
        Serial.print("failed, rc=");
        Serial.print(client.state());
        Serial.println(" try again in 5 seconds");
        delay(5000);
    }
}
}
// =====
// CALLBACK MQTT
// =====
void callback(char* topic, byte* payload, unsigned int length) {
    String msg;
    for (unsigned int i = 0; i < length; i++) {
        msg += (char)payload[i];
    }

    Serial.print("[MQTT] Topic: ");
    Serial.print(topic);
    Serial.print(" | Message: ");
    Serial.println(msg);

    if (String(topic) == "motor/throttle") {
        int throttle = msg.toInt();
        throttleLevel = constrain(throttle, 0, 3);
        switch (throttleLevel) {
            case 1: targetSpeedKmh = 5; break;
            case 2: targetSpeedKmh = 10; break;
            case 3: targetSpeedKmh = 15; break;
        }
        Serial.println("[MQTT] Throttle level set: " + String(throttleLevel));
    }

    if (String(topic) == "motor/direction") {
        msg.toLowerCase();
        if (msg == "forward" || msg == "reverse" || msg == "neutral") {
            motorDirection = msg;
            Serial.println("[MQTT] Direction set: " + motorDirection);
        }
    }
}

// =====
// SETUP
// =====
void setup() {

```

```

Serial.begin(115200);

pinMode(pinREn, OUTPUT);
pinMode(pinLEn, OUTPUT);

ledcSetup(0, 5000, 8); // 5kHz, 8-bit PWM
ledcAttachPin(pinMotorPWM, 0);

pinMode(encoderPinA, INPUT_PULLUP);
pinMode(encoderPinB, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(encoderPinA), encoderISR,
RISING);

setup_wifi();
client.setServer(mqtt_server, mqtt_port);
client.setCallback(callback);
}

// =====
// MENGHITUNG KECEPATAN
// =====
void updateSpeed() {
  noInterrupts();
  long count = encoderCount;
  encoderCount = 0;
  interrupts();

  float rpm = (count / 20.0) * 60.0; // 20 pulses per rotation
  float wheelCircumference = 3.14 * 0.1; // Diameter roda 10 cm
  actualSpeedKmh = (rpm * wheelCircumference * 60.0) / 1000.0;

  Serial.println("[ESP32] Actual Speed: " + String(actualSpeedKmh, 2) + "
km/h");
  client.publish("motor/actual_speed", String(actualSpeedKmh, 2).c_str());
}

// =====
// KONTROL ARAH
// =====
void controlDirection() {
  if (motorDirection == "forward") {
    digitalWrite(pinREn, HIGH);
    digitalWrite(pinLEn, LOW);
  } else if (motorDirection == "reverse") {
    digitalWrite(pinREn, LOW);
    digitalWrite(pinLEn, HIGH);
  } else { // neutral
    digitalWrite(pinREn, LOW);

```

```

        digitalWrite(pinLEn, LOW);
    }
}

// =====
// PID CONTROL
// =====
void PIDControl() {
    if (motorDirection == "neutral" || targetSpeedKmh == 0) {
        ledcWrite(0, 0);
        pidPWM = 0;
        integral = 0;
        lastError = 0;
        return;
    }

    error = targetSpeedKmh - actualSpeedKmh;
    integral += error;
    float derivative = error - lastError;
    lastError = error;

    float output = Kp * error + Ki * integral + Kd * derivative;
    pidPWM = constrain((int)output, 0, 255);
    ledcWrite(0, pidPWM);

    Serial.println("[ESP32] PID PWM: " + String(pidPWM));
    client.publish("motor/pid_output", String(pidPWM).c_str());
}

// =====
// LOOP UTAMA
// =====
void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    controlDirection();

    unsigned long now = millis();
    if (now - lastSpeedCheck >= 1000) {
        lastSpeedCheck = now;
        updateSpeed();
        PIDControl();
    }
}

```

ANALISIS

Penggunaan sistem kontrol Proporsional-Integral-Derivatif (PID) dalam pengaturan kecepatan motor DC efektif digunakan dalam mencapai kestabilan dan respons dinamis yang baik. Pada tugas ini, perancangan dan implementasi kontrol PID dilakukan menggunakan *software* Matlab untuk simulasi dan Arduino IDE untuk realisasi fisik berbasis mikrokontroler ESP32. Adapun beberapa tahapan yang dilakukan yaitu pada tahap pertama melakukan pemodelan sistem motor DC berdasarkan parameter listrik dan mekanik, yang meliputi resistansi (R), induktansi (L), momen inersia (J), gesekan (B), konstanta tegangan balik (Ke), dan konstanta torsi (Kt). *Transfer function* sistem *open-loop* disusun dari parameter tersebut, dan kemudian dikendalikan menggunakan kontroler PID yang telah disesuaikan dengan nilai gain proporsional (Kp), integral (Ki), dan derivatif (Kd) dari hasil *tuning*.

Hasil simulasi pada *software* Matlab menunjukkan bahwa kontrol PID yang dirancang mampu menghasilkan karakteristik respons sistem yang cepat dan stabil, ditandai dengan nilai *rise time* sebesar 0,0166 detik, *settling time* sebesar 0,0294 detik, *peak time* sebesar 0,0554 detik, dan *overshoot* sebesar 0,10%. Nilai-nilai tersebut menunjukkan bahwa sistem memiliki waktu tanggap yang sangat cepat dengan deviasi minimal terhadap nilai referensi, yang mencerminkan kinerja kendali yang optimal.

Implementasi kontrol PID secara fisik direalisasikan menggunakan ESP32 yang dikonfigurasi untuk menerima perintah dari antarmuka MQTT berbasis jaringan Wi-Fi. Kode program pada *software* Arduino IDE menunjukkan penggunaan pengendali PID dalam pengaturan kecepatan motor berdasarkan pembacaan sensor *rotary encoder*. Sistem ini mengatur kecepatan motor secara *real-time* melalui perhitungan error antara kecepatan aktual dan kecepatan target, serta menerapkan aksi PID untuk menyesuaikan sinyal PWM. Selain itu, sistem juga mendukung pengaturan arah putaran motor melalui MQTT, sehingga dapat mengontrol arah gerakan motor secara fleksibel (*forward*, *reverse*, dan *neutral*).

Selain itu, integrasi antara simulasi dengan menggunakan *software* Matlab dan implementasi mikrokontroler pada ESP32 menunjukkan keselarasan antara desain teoritis dan aplikatif. Efektivitas kontrol PID tidak hanya ditunjukkan dalam simulasi tetapi juga dalam praktik nyata, menjadikan pendekatan ini sangat relevan untuk diterapkan pada sistem otomasi dan robotika berbasis motor DC.